Comparison of Logistic Regression, Random Forest, XGBoost, Convolutional Neural Networks, and Multi-Layer Perceptrons in Classifying Encrypted and Non-Encrypted (VPN & Non-VPN) Network Traffic

Benji Kiblinger
rbkiblinger@wm.edu
William & Mary
Williamsburg, Virginia, USA

Abstract

The widespread adoption of encrypted internet protocols has made normal deep packet inspection techniques ineffective for modern network traffic analysis. Identifying the specific category of network traffic, without direct access to the URLs or the content of its payloads, requires models based off of statistical flow-level features. In this work, I present a comparison of machine learning models for classifying both encrypted (VPN) and non-encrypted (non-VPN) network traffic across seven application categories. These categories are as follows: web browsing, chat, file transfer, email, P2P, streaming, and VoIP. To this end, I used the ISCX VPN-nonVPN dataset [6] containing real network traffic from various applications including Skype, YouTube, BitTorrent, and others, and was captured in both standard and VPN-encrypted conditions. The dataset was made by extracting the flow-level features using the ISCXFlowMeter over various time intervals, from 15 seconds to 120 seconds. A logistic regression model was established as the baseline for this comparison, and it is used to evaluate against a Random Forest [3] model, XGBoost [5], a 1-dimensional Convolutional Neural Network (CNN), and a Multi-Layer Perceptron (MLP). For both encrypted and non-encrypted traffic, the main finding of this work is that XGBoost achieves the strongest performance, with a 90.4% accuracy and 0.87 macro F1-score on the 15-second all in one dataset. Random Forest follows closely behind at 88.9% accuracy and 0.86 macro F1-score, outperforming the logistic regression baseline by 34.4 percentage points in accuracy. The 1D CNN achieved an accuracy of 80.1% and a macro F1 of 0.75, while the MLP outperformed it at 82.3% accuracy and 0.77 macro F1, demonstrating that convolutional operations are poorly suited to tabular flow features that carry no spatial structure. Both deep learning models trailed the tree-based methods by a substantial margin. These results were gathered by analyzing the all in one dataset, which combines both VPN and non-VPN traffic into a single 7-class classification problem. These findings demonstrate that gradient boosting and ensemble tree methods substantially outperform deep learning on this tabular network flow dataset, consistent with the broader literature showing that tree ensembles tend to perform well on small, low-dimensional tabular data [7].

Keywords

network traffic classification, encrypted traffic, VPN, machine learning, random forest, convolutional neural network, multilayer perceptron, logistic regression, xgboost, flow level features, deep packet inspection, network analysis, ISCX dataset

1 Introduction

Network traffic classification and analysis is one of the most foundational aspects in network management, security monitoring, and ensuring high quality network access across various types of applications. Traditional network classification relies on deep packet inspection (DPI) which examines the payload content of the packets in the network to identify the application type or protocol. However, in the modern day, network traffic is becoming increasingly encrypted through the use of TLS and VPN tunneling, which renders deep packet inspection essentially ineffective, since the payload is now opaque to outside observers and the destination URL is often not available [1].

This creates a problem for the network manager: how can the traffic type be identified without using traditional deep packet inspection techniques? One answer is flow level statistical analysis, that is, using the aggregate measurements of a network flow, such as its packet sizes, arrival times, flow durations, and byte counts. These features are observable even if the payload is encrypted, and have been shown to be promising in distinguishing between the different categories of traffic [1, 6].

In this work, five models of increasing complexity are used to classify the categories of network traffic, using the ISCX VPN-nonVPN dataset [6]. This is a publicly available

dataset containing labeled flow level network traffic across seven application categories, including both standard and VPN-encrypted sessions. The five models, logistic regression, Random Forest [3], XGBoost [5], a 1-dimensional Convolutional Neural Network (CNN), and a Multi-Layer Perceptron (MLP), are compared using their accuracy percentages and their macro F1 scores. The CNN and MLP are included to investigate whether deep learning can compete with tree-based ensembles on this type of data, and to examine whether the spatial inductive bias of convolution is appropriate for tabular flow features.

The contributions of this work are:

- A complete classification pipeline from raw ARFF feature data to trained, evaluated models.
- A systematic comparison of a linear baseline, two ensemble tree methods, and two deep learning architectures on the same dataset.
- An empirical demonstration that the convolutional inductive bias is inappropriate for tabular flow features, and that an MLP is a more suitable deep learning baseline.
- Analysis of per-class performance to identify which traffic types are hardest to distinguish.

2 Proposed Method

2.1 Dataset

The dataset used in this work is the ISCX VPN-nonVPN dataset [4, 6]. This dataset, created by Draper-Gil et al. from the University of New Brunswick, was made by capturing live network traffic from real applications in both encrypted (VPN) and non-encrypted (no VPN) conditions. The traffic was generated from applications spanning seven distinct categories: web browsing (Chrome, Firefox), chat (Skype, ICQ, AIM, Facebook), file transfer (FTPS, SFTP), email (SMTPS, POP3S, IMAPS), P2P (Vuze, uTorrent), streaming (Vimeo, YouTube), and VoIP (Skype audio, Facebook audio). The raw packet captures were then processed using ISCXFlowMeter to extract the bidirectional flow-level statistical features over various fixed time windows from 15 seconds to 120 seconds. This work uses the 15 second variation, since shorter time windows produces approximately twice as many labeled flows as the 120 second window, yielding around 19,000 samples total compared to only around 9,000. This greatly improves the generalization across all models as they have more data to work with.

This dataset is then split into 80% training, 10% validation, and 10% test splits using stratified sampling to preserve the class proportions across all of the splits. Additionally, the class distribution of the data is imbalanced: BROWSING and VOIP each account for roughly 27% of samples, while MAIL (3.9%) and STREAMING (5.1%) are the smallest classes. This imbalance makes the macro F1-score the primary evaluation metric, as it weights all of the classes equally, regardless of their size.

2.1.1 Flow-Level Features. As mentioned prior, the primary way to do network traffic analysis without direct access to their payloads is by analyzing their statistical flow-level features. In this dataset, ISCXFlowMeter extracts 23 statistical features per flow, all of which are computed from the packet headers without needing to inspect the payload content itself. Table 1 lists all of the flow-level features grouped by type and the text below explain what each group captures and why they are discriminative enough to draw conclusions about the traffic.

Table 1: ISCXFlowMeter flow-level features

Group	N	Feature names
Flow duration	1	`duration`
Forward IAT	4	`total_fiat`, `min_fiat`, `max_fiat`, `mean_fiat`
Backward IAT	4	`total_biat`, `min_biat`, `max_biat`, `mean_biat`
Bidir. IAT	4	`min_flowiat`, `max_flowiat`, `mean_flowiat`, `std_flowiat`
Throughput	2	`flowPktsPerSecond`, `flowBytesPerSecond`
Active periods	4	`min_active`, `mean_active`, `max_active`, `std_active`
Idle periods	4	`min_idle`, `mean_idle`, `max_idle`, `std_idle`

Flow duration provides a coarse signal on its own: VoIP sessions are long-lived, while individual web requests such as that of browsing are short.

Forward and backward inter-arrival times (IAT) measure the time gap between consecutive packets in each direction. Directional IATs reveal the asymmetry present in communication patterns: file transfers tend to have small and regular forward IATs during bulk sending, while chat applications show signs of irregular and human dependent timing.

Bidirectional flow IAT is an aggregate of the inter-arrival time across both directions. The standard deviation in particular captures burst loads: P2P flows tend to be full of bursts, while VoIP flows have low standard deviation due to their constant-bitrate codec streaming behavior.

Throughput measures the rate at which packets and bytes are transmitted. This is among the most discriminative of features: streaming tends to generates high and sustained byte rates, VoIP generates low but consistent packet rates, and email generates low rates with a single large burst when the mail is pushed or pulled.

Active and idle periods capture the session-level behavior that is opaque in the per-packet statistics. ISCXFlowMeter segments each flow into active sub-flows and idle gaps, computing statistics over each. A web browsing session alternates between short active bursts (page loads) and long idle periods (reading), whereas something like a VoIP call has a single long active period with minimal idle time.

2.2 Pre-Processing and Experimental Setup

All five models share the same pre-processing pipeline to ensure fair comparison. All of the features are standardized using `StandardScaler`, which subtracts the per-feature mean and divides by the standard deviation of the feature. The scaler is then fit exclusively on the training set and then applied to the validation and test sets, which prevents any leakage of test statistics into training. Although the tree-based models are scale invariant, standardization is still applied uniformly so that differences in performance can be attributed to the model architecture rather than the pre-processing methods.

As previously mentioned, the dataset is split into 80% training, 10% validation, and 10% test splits using stratified sampling, which preserves the class distribution in each split. The validation set serves two purposes in the evaluation: it allows early stopping for the models that support it (namely the MLP and CNN), and a final sanity check on the model selection. The test split is not touched until the final evaluation is run, and is thus never used to make any modeling decisions, preventing the results from being tarnished. All of the models use a fixed random seed, set to 42, for reproducibility purposes.

2.2.1 Logistic Regression (Baseline). Logistic regression is used as the baseline method in this work because it is one of the simplest models capable of multiclass classification, and its performance thus establishes a lower bound for the more complex models to exceed in order to be useful. The model is trained using the `class_weight='balanced'`, which reweights all of the training samples inversely proportional to their class frequency, compensating for the imbalance present between smaller and larger classes in the dataset. The solver is then allowed up to 1,000 iterations to reach a convergence.

2.2.2 Random Forest. The Random Forest model [3] uses an ensemble of 200 decision trees, each of which are trained on a sample of the training data showing a random subset of features considered at each of the splits. (Defaulting to $\sqrt{23} \approx 5$ features). The predictions are made by a majority vote across all of the trees. As with logistic regression, the `class_weight='balanced'` is used to correct the imbalances present in the data between larger and smaller class sizes. Additionally, the ensemble size of 200 was chosen as a point of diminishing returns, having tested 300, 400, and 500, all of which made no significant difference to the results.

2.2.3 XGBoost. The XGBoost [5] model uses gradient boosted trees, which builds an ensemble sequentially with each new tree correcting the residual errors of the previous one. Additionally, the model is configured with 500 estimators, a maximum tree depth of 6, a learning rate of 0.1, and subsampling rates of 0.8 for both rows (`subsample`) and columns (`colsample_bytree`) to reduce overfitting of the data. Training uses the validation set for early stopping with a patience of 20 rounds. This means that the training will halt if the validation log-loss does not decrease (improve) for over 20 consecutive rounds (meaning the training hit a ceiling). Unlike the Random Forest model, XGBoost's boosting mechanism naturally focuses on previously misclassified examples at each round so explicitly balanced class weighting was found to be ineffective, and is therefore not applied.

2.2.4 1D Convolutional Neural Network. The 1D CNN model in this work treats the 23 feature vector as a sequeunce of length 23 with a single input channel, and then applies two convolutional layers to extract the local patterns. Both convolutional layers use `padding=1` to preserve the sequence length across the network. The architecture is thus: `Conv1d(1, 32, k=3, p=1)` → `ReLU` → `Conv1d(32, 64, k=3, p=1)` → `ReLU` → `Flatten` → `Linear(1472, 128)` → `ReLU` → `Dropout(0.3)` → `Linear(128, 7)`. The CNN is included primarily as a point of comparison, to demonstrate that the convolutional network's assumption of spatial correlations between adjacent inputs is not appropriate for a purely tabular dataset such as the one used in this work, where feature ordering is arbitrary.

2.2.5 Multi-Layer Perceptron. The MLP model used in this work uses three fully connected hidden layers, two of width 256, and one of width 128, each followed by a layer normalization [2], ReLU activation, and a dropout of 0.3, 0.3, and 0.2 respectively. Layer normalization is used for this instead of batch normalization, because it normalizes per-sample rather than per-batch, aiming to avoid the training/evaluation discrepancy arising from running statistics on small and imbalanced tabular datasets. The full architecture is thus: `Linear(23, 256)` → `LayerNorm` → `ReLU` → `Dropout(0.3)` → `Linear(256, 256)` → `LayerNorm` → `ReLU` → `Dropout(0.3)` → `Linear(256, 128)` → `LayerNorm` → `ReLU` → `Dropout(0.2)` → `Linear(128, 7)`.

2.2.6 Note on the Neural Networks. Both of the neural network models are trained using the Adam optimizer [8], a learning rate of 3×10^{-4}, weight decay of 10^{-4}, and gradient clipping at norm 1.0. The learning rate is following a cosine annealing schedule, decaying from 3×10^{-4} to 10^{-6} over

800 epochs. Early stopping with a patience of 40 halts the training, if the validation loss does not improve significantly over those 40 consecutive epochs, and the weights from the best validation epoch are restored before the evaluation.

2.3 Evaluation Metrics

All of the above models are evaluated using their accuracy and macro-averaged F1-score. The accuracy measures the fraction of correctly classified flows overall, while the macro F1 averages all of the class dependent F1-scores with equal weight. The macro F1 score is the primary metric simply because of the imbalance present in the dataset. A model that perfectly classifies BROWSING and VOIP while ignoring MAIL would acheive a high overall accuracy, but poor macro F1-score, thus misrepresenting how useful it would actually be for classifying real network traffic.

3 Evaluation

Table 2 shows the test set accuracy and macro F1-score for all five of the tested models. Among these, XGBoost achieves the highest overall performance with 90.4% accuracy and 0.87 macro F1, followed closely by Random Forest with 88.9% accuracy and 0.86 macro F1. Both of the deep learning models (CNN & MLP) trail the tree based methods by a significant margin. The MLP only reaches 82.3% and the 1D CNN reaches 80.1%. Additionally, the logistic regression baseline only scores a 54.5%, which confirms that this classification problem is too difficult for a linear model and thus that the gains from more complex architectures are significant.

Table 2: Performance across all five tested models

Model	Accuracy	Macro F1
XGBoost	90.4%	0.87
Random Forest	88.9%	0.86
MLP	82.3%	0.77
1D CNN	80.1%	0.75
Logistic Regression	54.5%	0.44

3.1 Tree-Based Models

Both of the tree-based models perform within 1.5% of each other on the test split (90.4% vs 88.9%). However, XGBoost has a consistent edge across nearly every class of traffic evaluated, as shown in Table 3. The most notable difference between the two is in the CHAT class, where XGBoost scores 0.81 F1 compared to Random Forest's 0.73. This gap is larger than the overall accuracy difference would suggest, indicating that XGBoost's sequential boosting/error correcting is better at recovering from the hard ambiguous examples produced by the CHAT class. Additionally, XGBoost's early stopping triggered at round 108 of a maximum 500, which means the model converged rather quickly. Random Forest, however, being a bagging method with no sequential dependency between trees, has no equivalent stopping criterion and thus trains all 200 trees regardless.

3.2 Deep Learning Models

The MLP outperforms the 1D CNN on every one of the traffic classes, with an overall accuracy gap of 2.2 percentage points (82.3% vs 80.1%). This result is consistent with the fact that convolution is not a good choice for tabular data; since the 1D CNN imposes a local receptive field over the feature vector and the 23 flow level features have no spatial relationship with each other. The MLP treats each of these features as an independent feature through its fully connected layers, and thus learns without the structural constraint, achieving better generalization as a result.

The MLP reached an early stopping point at epoch 515 out of a maximum 800, and the CNN also stopped at a similar point. Both of these models showed a smooth monotonically decreasing validation loss after switching from batch normalization to layer normalization which previously caused instability.

3.3 Per-Class Performance

Table 3 shows the F1-score for each traffic class across all of the five tested models.

As shown in the table, VOIP is the easiest class for all of the models to classify across the board, reaching a 0.92–0.99 F1-score in every model. This is because of its traffic pattern, which consists of small, constant-bitrate packets, all of which have very low inter-arrival time deviance. This is very distinctive in the feature space, and separable even by the simplest model (logistic regression). Comparatively, CHAT is consistently the hardest class, with an F1-score ranging from 0.36 in the baseline, to 0.81 in the XGBoost model. File transfer is also one of the weakest classes, with a 0.23 F1-score in the baseline and recovering strongly in the two tree based models, 0.82 & 0.84. This suggests that its features are non-linearly separable, in a way such that logistic regression cannot capture it. These trends will be discussed further in Section 4.

3.4 Feature Importance

Due to the fact that the Random Forest and XGBoost models are the overall strongest performing models, their feature importances will offer the most meaningful insights into which of the flow-level features are most important for the task of classifying the traffic. Both of the models compute the feature importance as the mean decrease in impurity across

Table 3: Per-class F1-score on the test set

Class	Log. Reg.	CNN-1D	MLP	Rand. Forest	XGBoost
BROWSING	0.55	0.83	0.85	0.91	0.93
CHAT	0.36	0.55	0.63	0.73	0.81
FT	0.23	0.70	0.74	0.82	0.84
MAIL	0.26	0.72	0.74	0.86	0.88
P2P	0.47	0.71	0.73	0.88	0.88
STREAMING	0.32	0.73	0.74	0.82	0.81
VOIP	0.92	0.98	0.98	0.99	0.99

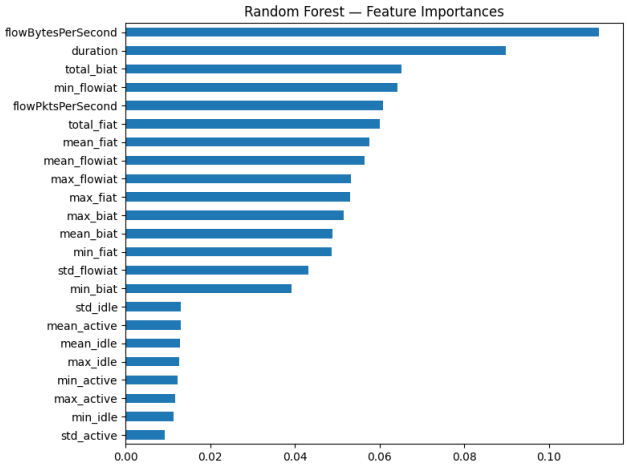

Figure 1: Random Forest Feature Importances

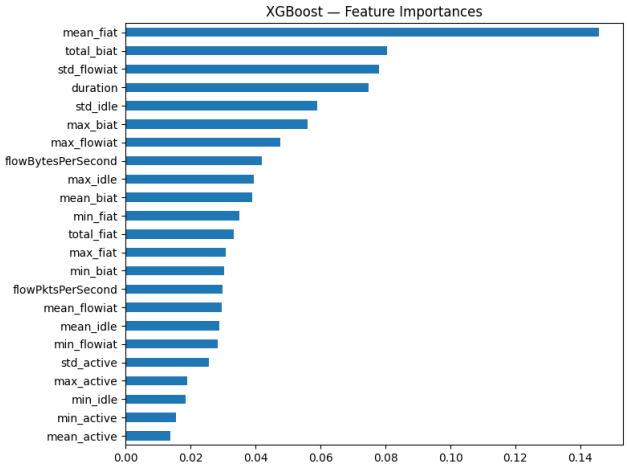

Figure 2: XGBoost Feature Importances

all of their trees. Figures 1 and 2 show all of the ranked feature importances and the key findings will be summarized below.

Both of the models agree on several highly important features, but notably, they differ on which one is the single most important one. For Random Forest, `flowBytesPerSecond` is the most important (importance 0.112), followed by `duration` (0.090) and `total_biat` (0.065). For XGBoost, `mean_fiat` ranks first by a wide margin (0.146), with `total_biat` (0.081) and `std_flowiat` (0.078) following. This disagreement between the two models shows the architectural differences present in them. Where Random Forest evaluates all features with equal opportunity, across independently grown trees, XGBoost's sequential boosting tends to concentrate the importance on whichever features resolve the residual errors of previous rounds of training the most efficiently.

Despite this disagreement of the most important features, both of the models consistently rank the IAT-based features highly as well. `total_biat`, `mean_fiat`, `max_biat`, and the bidirectional flow IAT statistics all are in the top half of both of the rankings. This shows that the timing of packets; how often they arrive, and in which direction, is the primary distinguishing features of the traffic types. Meanwhile, `std_flowiat` captures the burstiness of a flow, as P2P traffic tends to exhibit high variance as peers exchange data in irregular chunks, while VoIP maintains near-constant IATs due to the codec timing. `duration`, however, ranks highly in both of the models as well, since the session lengths differ across the various categories.

Compared to this, the active and idle period statistics are consistently among the least important features in both of the models. All eight of the active/idle features rank in the bottom third of the two rankings. The one exception is `std_idle` (XGBoost rank 5, importance 0.059) and `max_idle` (XGBoost rank 9), suggesting that the variance and maximum of idle gaps does in fact carry some importance. This is likely because something such as browsing sessions produce distinctively long idle gaps between page loads. The minimum values of nearly all of the feature groups (`min_active`, `min_fiat`, `min_biat`) rank at the very bottom, as they are dominated by the initialization noise present in nearly every traffic type.

4 Discussion

4.1 Interpretation of Results

The overall finding of this work is that the gradient boosted and ensemble based tree methods substantially outperform the deep learning on this specific, tabular, dataset. This result is consistent with the broader literature on machine learning, where tree ensembles are shown to be very competitive with the deep learning on small low-dimensional tabular datasets [7]. The main reason for this is because that the deep learning models are primarily designed to learn complex hierarchical representations from very large amounts of data, and neither of those conditions are met with the ISCX-VPN-nonVPN dataset. With around 15,000 training samples and only 23 features, XGBoost an Random Forest can reach near-optimal decision boundaries in the feature space, with far fewer parameters than something like a neural network would require. Overall, for this specific classification task, there is just not enough data in the dataset to justify their increased complexity and training times.

The failure of the 1D CNN in particular is quite insightful. Since convolutional neural networks were designed with the assumption that nearby inputs are somehow spatially correlated, datasets without these spatial correlations like this tabular dataset are not suited for CNNs. Additionally, the 23 flow-level features present in the dataset also do not carry any such spatial structure: `duration` sits next to `flowBytesPerSecond` in the feature vector not because they are related, but purely because of how ISCXFlowMeter arbitrarily orders its output. Thus, the CNN therefore wastes its receptive field on meaningless local neighborhoods, while the fully connected MLP at least has the freedom to learn from arbitrary cross-feature interactions. The 2.2 percentage point gap between the MLP and CNN directly reflects this fact.

The per-class resulst also reveal several meaningful patterns. VOIP is shown to be the easiest class for every model, with even the logistic regression baseline achieving a 0.92 F1-score. This makes intuitive sense because VoIP codecs transmit at a fixed rate which produces flow patterns with nearly constant IATs and a low variance. These features are very distinctive, and separate VOIP cleanly from every other traffic class. CHAT, however, is consistently the hardest class across all models, ranging from a 0.36 in the baseline to a 0.81 in XGBoost. An explanation for this is that the CHAT category aggregates several different chat applications, including Skype, ICQ, AIM, and Facebook, each of which have differing underlying transport-layer protocols and therefore will exhibit different timing and throughput characteristics. This interal heterogeneity makes CHAT a noisy class to draw conclusions from as opposed to one such as VOIP that maps cleanly to a single protocol behavior.

The feature importances also add further context to these results. Both of the tree-based models consistently show IAT features as the most important overall for classification, but they disagree on which singular feature is the most important. Random Forest ranks `flowBytesPerSecond` first, whereas XGBoost ranks `mean_fiat` first. This disagreement is one that is architectural in nature rather than arbitrary. Due to the fact that Random Forest trains the trees independently on random feature subsets, the importance scores thus reflect how often a feature was available and how much it reduced the impurity across all 200 trees. XGBoost, however, trains its trees sequentially and concentrates on errors that are present after each generation, it scales up the weights of whatever feature best resolves the remaining misclassifications at each round. The two views are thus complementary: `flowBytesPerSecond` is a globally reliable separator, while `mean_fiat` is particularly decisive in the hard cases that the earlier rounds may have missed.

One key aspect of this dataset is that the AllinOne variant (used in this work) combines both the VPN-encrypted and non-VPN traffic under the same seven application labels. This means that the models are not simply classifying traffic by protocol, but are classifying it by application category regardless of whether the underlying connection is tunneled through a VPN (encrypted) or not. Thus, the fact that the XGBoost model acheives a 90.4% accuracy on this aggregated dataset demonstrates that flow level statistical features are indeed enough to classify network traffic categories even despite them being encrypted or non-encrypted. This is primarily because despite the fact that encryption obscures the payload content and destination of the packets, the underlying timing and throughput behavior of the application are still present no matter what. A streaming session through a VPN still generates the same high and sustained byte rates as that of a non-VPN session; and a VoIP call through a VPN still produces constant low-variance inter-arrival times. The encryption obscures purely the content, but not the behavioral fingerprint of the application generating them, and thus this means that the models are able to learn to classify both the encrypted and non-encrypted network traffic.

A notable limitation of this work is that the ISCX VPN-nonVPN dataset was collected around 2016 and thus has captured traffic from application categories that have since changed quite significantly. Modern applications now mainly use TLS 1.3, QUIC, and HTTP/3, which alter the observable flow statistics compared to what was present in the dataset used [1]. Furthermore, because the dataset was generated in a controlled lab environment, with only one application category present at a time, it may not reflect the full unpredictability of real-world network traffic, where things such as background noise, retransmissions, and multi-application

sessions are omnipresent. Additionally, the choice of the 15-second time window is also limiting, and real deployments would need to confidently handle flows of unknown and/or variable length.

4.2 Future Work

The next step of this work is to deploy the trained models in a real pipeline to classify live network traffic. The models were trained on an entirely offline and pre-extracted ARFF dataset, so the main way to extend this work would be to deploy it in a real network classification pipeline. One such pipeline could capturing the real data, extracting the same 23 ISCXFlowMeter features in real time, and then feeding those to the trained XGBoost model for classification. This could be implemented using `tcpdump` or `tshark` to capture the raw packets, running ISCXFlowMeter on a 15 second rolling time window, and then feeding that to the trained model as each window closes. All of this could be baked into one python script, which could then output the model's most likely classification type for each window. Such a script would allow network managers to classify live traffic on their networks by application category without ever inspecting the payload content, which is valuable in VPN-encrypted environments and in contexts where privacy must be preserved.

Beyond this pipeline deployment several modeling improvements are worth exploring. As mentioned previously, the models in this work are trained on a single 15-second time window. An ensemble of training across all of the window sizes (15s, 30s, 60s, 120s) could reduce the model's sensitivity to flows that happen to cross a window boundary. Additionally, it would be worth inlcuding more recent traffic datasets such as ones that include the QUIC and HTTP/3 protocols to allow the same methodology to be re-evaluated against modern traffic. The reason this was not done in the first place is because the 2016 dataset was the most recent one able to be found for free during the time of writing. One final refinement could be to split up the CHAT category to capture the distinct protocols across the distinct applications, which may substantially improve the classification performance.

5 Conclusion

This work presents a comparison of five machine learning models in the task of classifying network traffic, across seven application categories trained on the ISCX-VPN-nonVPN dataset. The five models tested range from a baseline of logistic regression, ensemble tree methods, and finally deep learning architectures, in order of complexity. All of these models are evaluated on the same 15-second AllinOne dataset, using the accuracy and macro F1-score as the primary metrics. The results of this work show that XGBoost achieves the strongest overall performance of 90.4% accuracy and a 0.87 macro F1-score, with Random Forest trailing close behind at an accuracy of 88.9% and macro F1 of 0.86. As well as this, both of the deep learning models trail significantly behind, with the MLP at 82.3% and the 1D CNN at 80.1%, with the latter reflecting the fact that convolutional operations for tabular flow features with no spatial structure are not suitable enough to classify the application categories. Finally, the logistic regression baseline at 54.5% confirms that the problem is too non-linear for a simple model, thus validating the gains from the more complex tree-based architectures.

One of the main conclusions of this work is that the flow-level statistical features of network traffic alone are enough to classify the network traffic by application category. This is primarily because while encryption may obscure the payload content itself, the overall behavioral fingerprint of the network traffic remains unchanged and these features alone were proven to be sufficient and fundamental in classifying the network traffic. This work thus has significant implications for network management, as it demonstrates that even when traditional deep packet inspection is impossible due to VPN tunneling/encryption, meaningful classification is still achievable purely based on the flow-level statistical features, such as its timing patterns, byte rates, and overall session structure. Thus, the trained models and classification pipeline of this work provide a foundation for the live traffic classification system as described in the Future Work section.

References

[1] Ibrahim A. Alwhbi, Cliff C. Zou, and Reem N. Alharbi. 2024. Encrypted Network Traffic Analysis and Classification Utilizing Machine Learning. *Sensors* 24, 11 (2024), 3509. doi:10.3390/s24113509

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. *arXiv preprint arXiv:1607.06450* (2016).

[3] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[4] Canadian Institute for Cybersecurity. 2016. *VPN-nonVPN Dataset (ISCXVPN2016).* https://www.unb.ca/cic/datasets/vpn.html University of New Brunswick.

[5] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).* ACM, San Francisco, CA, USA, 785–794. doi:10.1145/2939672.2939785

[6] Gerard Draper-Gil, Arash Habibi Lashkari, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani. 2016. Characterization of Encrypted and VPN Traffic using Time-related Features. In *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP).* SciTePress, Rome, Italy, 407–414.

[7] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data?. In *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, Vol. 35.

[8] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR).* San Diego, CA, USA.